GeoCool: An Explainable GeoAI Decision-Support System for Urban Cooling Corridors

Team: Rajat, Ruchi, Janhavi (3 members) **Target Area:** Nagpur, Maharashtra (semi-arid, high summer heat) — scalable to other cities

---

1. Project Overview & Objective

Goal: Build a full-stack, AI-driven mapping platform that automates the routing of green corridors for city planners. It finds the mathematically optimal path that balances extreme urban heat, traffic pollution, physical road feasibility, and (added back) explains that route using real model-based XAI rather than only rule-based scoring.

---

2. Core Logic — Combining the 3 Reference Papers + ML Layer

Your routing engine operates on four combined frameworks. This is your one-sentence answer to "why this method" / "what's novel":

Source	What it contributes
Chennai highway-edge paper	Per-road-segment suitability scoring pattern (Tier 1/Tier 2 classification)
Adama City paper	AHP + weighted overlay method to turn raw layers into a suitability score
Amman green-roof paper	Multi-scenario weighting + "low-regret zone" validation, so weights aren't arbitrary
ML/SHAP layer (restored)	A trained Random Forest/Gradient Boosting model predicts suitability from the same feature table, and SHAP explains *why* — this is what actually justifies calling the system "AI," not just "GIS + weighted math"

Important — decide and state this explicitly to your guide:

- **AHP** = your primary, defensible, rule-based scoring method (mathematically verified via Consistency Ratio ≤ 0.1). This drives the route selection.
- **ML + SHAP** = a secondary, learned model trained on the same feature table, used as (a) a cross-validation check — "does a model trained on this data agree with our hand-weighted AHP scores?" — and (b) the source of natural-language

XAI explanations per segment, since SHAP values convert into sentences like *"prioritized due to high heat exposure and low existing vegetation."*

- This combination is what makes the "Explainable GeoAI" title in the title accurate — AHP alone is not AI, it's MCDA.

Feasibility vs. Need Engine (kept as-is): Balances *Need* (Heat + Pollution + Population) against *Feasibility* (Street Width + Flat Elevation + Water Access) — added because focusing purely on the hottest spots fails in a semi-arid climate where trees can't survive without adequate feasibility conditions.

Typology & NBS Matching (kept as-is): Evaluates the winning route across 4 dimensions (Width, Accessibility, NDVI, Urban Context) to assign a specific Nature-Based Solution package (Canopy Trees, Bioswales, Vertical Gardens, Phytoremediation belts, etc.) instead of a generic "plant a tree" output.

Note: this needs its own small rule set/decision table — none of the three reference papers directly maps width/accessibility/NDVI/context to specific NBS products. Build or cite this mapping explicitly; don't imply it comes straight from Chennai's paper.

---

3. Data Layers & Parameters

Layer	**What it measures**	**Source**	**Tech Integration**
Heat Index (LST)	Street-level temperature / Urban Heat Islands	USGS Landsat 8/9 Level 2 Tier 1	Google Earth Engine Python API (peak summer filter)
NDVI (restored)	Existing vegetation/greenery — needed both as a suitability criterion *and* as an NBS-matching input	Sentinel-2	Google Earth Engine Python API
Air Quality (PM)	PM 2.5/10 — traffic/industrial pollution	CPCB India API or WAQI	Live REST API call via Spring Boot
Population Density	Concentration of citizens, socio-economic impact	WorldPop (100m grid)	Google Earth Engine Python API
Street Network	Road geometries, widths, industrial zones	OpenStreetMap (OSM)	Overpass API / Python OSMnx

Layer	What it measures	Source	Tech Integration
Elevation (Slope)	Steepness — trees need 0–5% flat slope	SRTM DEM	Google Earth Engine Python API
Soil & Water	Moisture retention, soil type, plant survival	FAO Digital Soil Map / ISRIC SoilGrids	Google Earth Engine Python API

---

4. Tech Stack

Layer	Technology	Role
Frontend	React + Vite	UI, state management
	Mapbox GL JS (or React-Leaflet as fallback)	Interactive 3D map, scenario toggles, route drawing
Backend	Java + Spring Boot	REST API, orchestration; live PM API calls
	Maven	Build tool
Database	MySQL + Spatial extension	Projects, coordinates, geometries, species DB, results
AI/GIS Service	Python (FastAPI wrapper)	Core processing engine
	earthengine-api	Satellite data pulls (LST, NDVI, population, elevation, soil)
	GeoPandas	Geospatial joins, buffers, zonal stats
	NetworkX	Road graph → A*/Dijkstra route optimization
	Rasterio	Raster layer processing
	Scikit-learn	ML model (Random Forest/Gradient Boosting) — **restored**
	SHAP	Explainability for ML predictions — **restored**

Layer	Technology	Role
	NumPy / Pandas	General data wrangling
	AHP/MCDA (custom Python)	Pairwise comparison matrix, eigenvector weights, consistency ratio
Dev/Deploy	Git + GitHub	Version control
	Docker	Containerize Python microservice (optionally all services)

---

5. Final Solutions Delivered to the User

1. **Optimal Route** — glowing route line on the map, avoiding steep hills, prioritizing high-heat/pollution/population streets.
2. **XAI Justification** — plain-English explanation generated from real SHAP output on the trained model (not templated text).
3. **Context-Aware NBS Packages** — specific infrastructure recommendation per street.
4. **Drought-Resilient Botanical Specs** — MySQL-driven native species recommendations (Neem, Karanj, Bougainvillea, etc.) based on soil layer.
5. **Financial & Carbon Dashboard** — estimated project cost + net-zero/carbon-negative crossover tracker comparing construction emissions to species sequestration rates.

Scope flag: items 3–5 above are new relative to your original 12-week plan. Budget explicit time for building/citing a real NBS rule set, a species-to-soil lookup table, and a carbon math methodology with real sequestration-rate data — otherwise these get rushed or faked near the deadline.

---

6. Evaluation Plan (restored — was missing from the new draft)

Guides weight this section heavily. Run and report on:

1. **Shortest-path vs. optimized-route comparison** — quantify the trade-off (distance added vs. suitability gained).
2. **Sensitivity analysis** — vary your top 2 AHP weights and see how much the chosen route changes.

3. **Multi-scenario robustness test** — Ecology-First vs. Budget-First vs. balanced scenario; check if a "low-regret zone" (Amman-paper concept) holds across scenarios.
4. **AHP-vs-ML agreement check** — does the trained ML model's ranking of segments broadly agree with the AHP scores? Disagreement is worth discussing, not hiding.

Turn each into a table or map figure for the final report.

---

7. Team Roles (3 members — scope has grown, so this load is heavier per person than the original plan)

Person	Responsibility
Person A (heaviest load — core GIS + AI)	Data acquisition, AHP suitability model, road graph + routing (NetworkX), ML + SHAP layer, NBS typology matching rule set, carbon/financial calculations
Person B	Spring Boot backend, MySQL Spatial schema (incl. species DB), live PM API integration
Person C	React + Mapbox/Leaflet frontend, scenario toggles, XAI/dashboard display

Flag: with the new scope (NBS matching, species DB, carbon/financial dashboard, ML+SHAP all now on top of the original AHP+routing work), Person A's load has grown significantly and is now genuinely too much for one person to carry alone in the original timeline. Two realistic options:

- **Split Person A's work across all three** for specific weeks (e.g., everyone pitches in on data collection in Week 1, or Person B/C temporarily help with NBS rule-set research since it's more of a lookup-table/domain-research task than a coding-heavy one), or
- **Cut scope** — e.g., treat the carbon/financial dashboard and species DB as a "stretch goal" documented as future work, not a core deliverable, if week 10-12 timelines get tight. Decide this explicitly as a team rather than letting it default to Person A quietly falling behind.

---

8. Build Order (restored — sequence, not just task list)

Golden rule: prove Python logic standalone before wrapping in FastAPI, before Spring Boot, before frontend. Debug one function at a time.

1. Repo + env setup (Vite, Spring Initializr, Python venv, MySQL Docker)
2. Leaflet/Mapbox map on screen with click-to-select points
3. Road network as NetworkX graph — prove shortest-path works standalone
4. Pull data layers (LST, NDVI, PM, population, slope, soil), zonal-stat onto road segments
5. AHP suitability score + consistency ratio per segment
6. Weight graph edges by suitability, generate optimized route vs. plain shortest path
7. Train ML model on feature table, validate (RMSE/R^2), run SHAP, generate explanation strings
8. Build NBS matching rule set + species/carbon lookup logic
9. Wrap steps 3–8 in a FastAPI microservice
10. Spring Boot + MySQL Spatial (parallel, Person C) — mocked data first, real calls once step 9 works
11. React + Map frontend (parallel, Person D) — build against agreed API contract, wire to mock then real API
12. Integration — full flow, fix coordinate-order/CORS mismatches
13. Evaluation scripts (Section 6)
14. Documentation

---

9. Free vs. Paid — Every Tool/API in This Stack

Tool / API	Free tier	Paid tier / when you'd pay	Notes
USGS Landsat 8/9	Fully free	—	Public domain via USGS/GEE

Tool / API	Free tier	Paid tier / when you'd pay	Notes
Sentinel-2 (NDVI)	Fully free	—	Via Copernicus or GEE
Google Earth Engine (GEE)	Free for noncommercial/academic/research use — verify eligibility with your institution	Required for commercial/production use (billing account + Cloud pricing)	<cite index="46-1,46-2">Since April 2026, noncommercial projects get a recurring monthly quota of free compute (EECUs); the default "Community" tier gives enough for student-scale projects</cite>. As a student project this qualifies as noncommercial — register with your institutional email.
WorldPop	Fully free	—	Open data, no key needed for most downloads

Tool / API	Free tier	Paid tier / when you'd pay	Notes
OpenStreetMap / Overpass API / OSMnx	Fully free	—	Rate-limited on public Overpass servers; self-host if you hit limits
SRTM DEM	Fully free	—	USGS/NASA public data
FAO Soil Map / ISRIC SoilGrids	Fully free	—	Open data
CPCB India API	No standardized public REST API for arbitrary coordinates — check current CPCB CCR portal access before committing to this in your architecture	—	Higher-risk assumption; confirm access works in Week 1
WAQI (World Air Quality Index)	<cite index="21-1,11-1">Free — a valid token is required, with a default quota of 1,000 requests per second</cite>	—	Recommended as your primary PM source over CPCB given easier programmatic access
Mapbox GL JS	<cite index="34-1,35-1">50,000 free web map loads per month; a map load is counted each time the Map object initializes, and users can pan/zoom/toggle layers without extra charges within a 12-hour session</cite>	<cite index="36-1">Roughly $0.50–$5 per additional 1,000 loads beyond the free tier</cite>	Requires a credit card to activate even the free tier. A student demo won't come close to 50k loads/month.

Tool / API	Free tier	Paid tier / when you'd pay	Notes
React-Leaflet (alternative to Mapbox)	Fully free, open-source, no card needed	—	Lower-risk fallback if you don't need true 3D or don't want to add a card
Java Spring Boot, Maven	Fully free, open-source	—	
MySQL Community Edition	Fully free	MySQL Enterprise (not needed for this project)	
Python libraries (GeoPandas, NetworkX, Rasterio, Scikit-learn, SHAP, NumPy, Pandas, OSMnx)	Fully free, open-source	—	
FastAPI	Fully free, open-source	—	
Docker	Free for personal/student/small-team use (Docker Desktop)	Docker Business tier (not needed here)	
Git + GitHub	Free tier includes free private repos	GitHub Pro/Team (not needed)	Ask about GitHub Student Developer Pack for extra perks
Hosting (if you deploy the final demo online)	Free tiers exist (Render, Railway, Vercel, GitHub Pages for frontend) with sleep/limits	Paid if you need always-on backend + DB for the final demo	Only relevant if your guide wants a live deployed link,

Tool / API	Free tier	Paid tier / when you'd pay	Notes
			not just a local demo

Bottom line on cost: the entire project can be built and demoed at **zero cost** if you (a) confirm GEE noncommercial/academic eligibility, (b) use WAQI instead of CPCB for pollution data, and (c) either stay on Leaflet or keep Mapbox usage well under 50k loads/month for the demo. The only place real money could enter the picture is if you deploy a public, always-on hosted version for the final demo — and even that has workable free tiers for a short-lived student project.